

Flashcards & Quiz Desktop Application

Project Report

A Python & Tkinter desktop application for account-based flashcard study and quizzing, with interchangeable PostgreSQL and SQLite database backends.

Prepared by: Punit Yadav
June 2024

1. Introduction

The Flashcards & Quiz application is a desktop study tool built with Python and Tkinter. It allows individual users to create personal accounts, organize study material into named flashcard sets, and reinforce learning through two complementary study modes: a self-paced **Learn** mode and a scored **Quiz** mode. The project was developed as a practical exercise in combining a graphical user interface with persistent relational data storage, and it supports two interchangeable database backends — PostgreSQL and SQLite — so the same feature set can be demonstrated against either a client-server database or a lightweight embedded database.

1.1 Objectives

- Provide secure, per-user accounts so that flashcard sets and quiz history remain private to each learner.
- Allow users to author their own study material (word/definition flashcard pairs) rather than relying on fixed, pre-built content.
- Reinforce recall through an interactive quiz mode with real-time, on-screen scoring.
- Demonstrate sound relational schema design, including foreign-key relationships between accounts, flashcard sets, individual cards, and quiz history.
- Support two database engines (PostgreSQL and SQLite) behind an equivalent feature set, illustrating portability between a server-based and an embedded database.

1.2 Technology Stack

Layer	Technology Used
Programming language	Python 3
GUI framework	Tkinter, ttk, and ttkbootstrap ("superhero" theme)
Relational databases	PostgreSQL (via psycopg2) and SQLite 3
In-app reference browser	tkinterweb (embedded HTML widget)
Password security	PBKDF2-HMAC-SHA256 hashing with a unique per-user salt
Process communication	Session handoff file (name.txt) between login and main-application processes

2. System Architecture

The application is organised as two cooperating Python scripts per database backend: an **Account module** that owns sign-up and login, and a **main-application module** that owns flashcard management, Learn mode, and Quiz mode. Rather than sharing a single Tkinter root window, each module runs as its own operating-system process — the Account module launches the main application once a login succeeds, and the main application can launch the Account module again on logout.

2.1 Process Overview

Module	PostgreSQL Backend	SQLite Backend	Responsibility
Account module	Account.py	Account1.py	Sign-up, login, credential verification, session creation
Main application	Edu.py	Edu1.py	Flashcard sets, Learn mode, Quiz mode, account/history view
Shared utility	auth_utils.py	auth_utils.py	Password hashing and verification (shared by both Account modules)

2.2 Session Handoff

Because the login window and the main application run as separate processes, session state cannot simply be held in memory. After a successful login, the Account module writes the authenticated username and account ID to a small text file (`name.txt`) and launches the main application as a subprocess. The main application reads this file on startup to identify the active user, and rewrites it on logout to clear the session before returning control to the Account module.

2.3 Database Schema

Both backends implement an equivalent relational schema (using engine-appropriate syntax, e.g. `SERIAL` in PostgreSQL versus `AUTOINCREMENT` in SQLite):

Table	Key Columns	Purpose
Accounts	acc_id (PK), name (unique), city, phone_number, password (hash)	One row per registered user
History	h_id (PK), acc_id (FK → Accounts), quiz_topic, Scores	Records a completed quiz attempt for a user
flashcard_sets	id (PK), name, ac_id (FK → Accounts)	A named, user-owned collection of flashcards

Table	Key Columns	Purpose
flashcards	id (PK), set_id (FK → flashcard_sets), word, definition	An individual word/definition flashcard within a set

2.4 Architecture Diagram

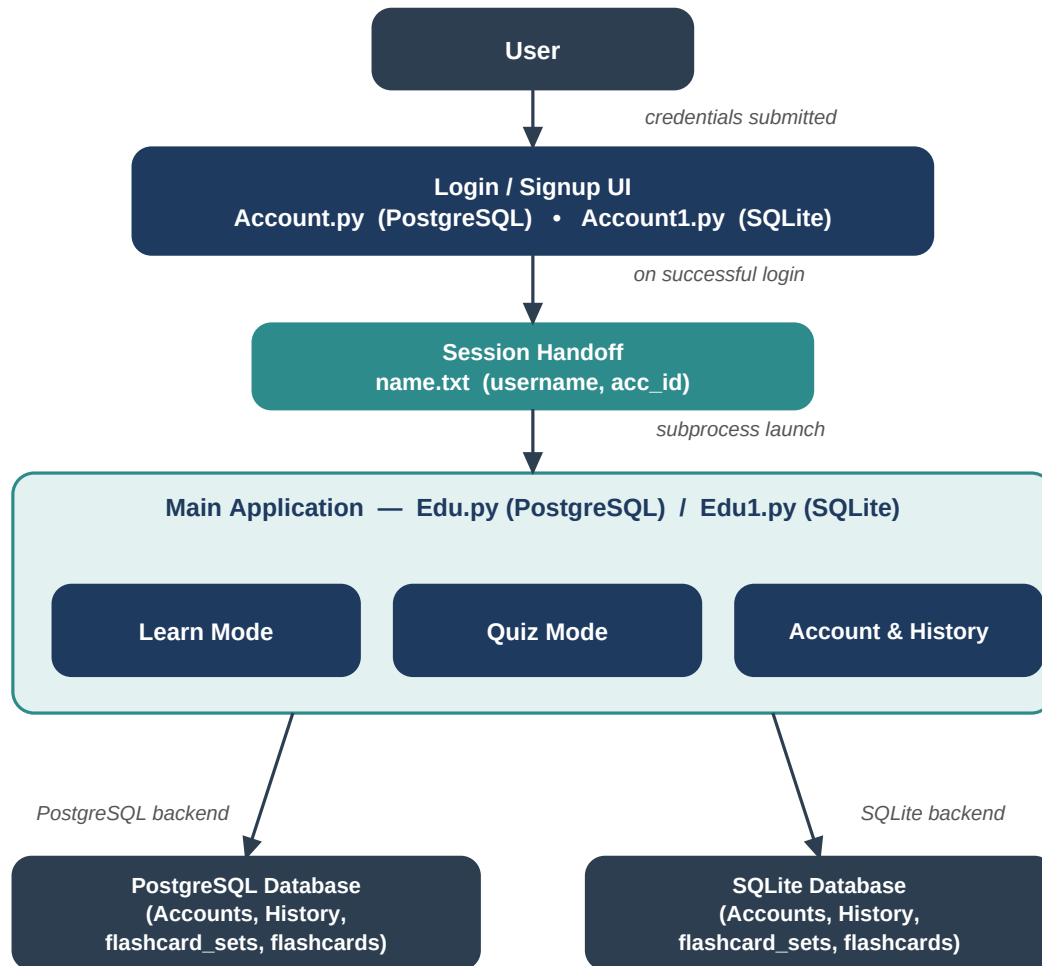


Figure 1. Login/session flow and the two parallel database backends.

2.5 Security Layer

Both Account modules delegate password handling to a shared `auth_utils.py` module. Passwords are never stored or compared as plain text: each password is combined with a randomly generated, per-user salt and run through PBKDF2-HMAC-SHA256 before being written to the password column, and the same procedure is repeated on login to verify the entered password against the stored hash.

3. Functionality Implemented

3.1 Account Management

- Sign-up form collecting name, city, phone number, and password (with re-typed password confirmation and required-field validation).
- Duplicate-username prevention — the username column is unique, and a clear error is shown before any database write is attempted.
- Passwords are salted and hashed before storage and are verified the same way at login, rather than being stored or compared in plain text.
- Successful login writes a session file and hands off to the main application; logging out clears the session and returns the user to the login screen.
- If a database error occurs during sign-up, the transaction is rolled back so the database is left in a consistent state.

3.2 Flashcard Set Management

- Create new, named flashcard sets that are scoped to the logged-in account.
- Add word/definition flashcard pairs to a chosen set.
- Select an existing set from a drop-down to study or quiz on its cards.
- Delete a set, with a confirmation prompt before the underlying cards are removed.

3.3 Learn Mode

- Sequential, self-paced browsing through the cards in a selected set.
- Each card displays its word and definition so the user can review material at their own pace before attempting the quiz.

3.4 Quiz Mode

- Presents each card's word and accepts a typed answer for its definition.
- Answers are checked case-insensitively against the stored definition.
- A running score ("Score: x / y") is updated and displayed after each answer.
- The quiz can be restarted at any time for the currently selected set.

3.5 Supplementary Features

- An embedded in-app browser tab (via tkinterweb) for quick reference look-ups while studying, without leaving the application.
- A consistent visual theme (ttkbootstrap “superhero”) applied across both the login window and the main application window.
- Equivalent functionality is available on both the PostgreSQL and SQLite backends, so either can be used interchangeably depending on the deployment environment.